

BERTopic for R: An Interface to the Python Implementation for Behavioral Research Workflows

Biying Zhou¹, Yuhao Yuan¹, Nanxin Deng², and Feng Ji¹

¹[TO DO: Affiliation]

²[TO DO: Affiliation]

[TO DO: Corresponding author name, postal address,
telephone number, and email]

Abstract

BERTopic provides an embedding-based approach to topic modeling. We describe BERTopic, an open-source R interface that retains fitted Python models and exposes S3 methods, tibbles, and matrices. An SMS example covers setup, fitting, inspection, prediction, visualization, export, and persistence. Five paired fresh-process runs compared selected R and Python outputs using 2,247 documents and frozen five-dimensional reduced embeddings, with identically configured components and process settings. The two paths returned identical assignments, metadata, and ordered term keys; weights and membership-strength matrices agreed within an absolute tolerance of 1e-12. In this controlled Windows task, paired R-minus-Python differences had medians of 4.183 seconds for fitting, 11.400 seconds for cold-process time, and 200.0 MiB for peak memory. Embedding generation and UMAP fitting were excluded from these workflow measurements. The contribution is an R-oriented access layer and a validation of its tested fitting and extraction path, not a new topic model or an independent implementation of BERTopic.

Keywords: topic modeling; BERTopic; R package; Python interoperability; behavioral text analysis; research software

1 Introduction

Behavioral researchers use open-ended survey responses, interviews, online discussions, clinical notes, and administrative records to study information that may be difficult to capture with closed-response measures. Exhaustive manual coding becomes costly as these collections grow. Topic models provide one way to summarize recurring patterns and construct document-level representations for subsequent qualitative inspection or quantitative analysis (Blei et al., 2003; Grimmer & Stewart, 2013; Roberts et al., 2014).

Classical methods such as latent semantic analysis, probabilistic latent semantic analysis, non-negative matrix factorization, and latent Dirichlet allocation represent documents primarily through word occurrence patterns (Blei et al., 2003; Deerwester et al., 1990; Hofmann, 1999; Lee & Seung, 1999). Transformer models instead provide contextual representations in which documents with different surface forms can occupy nearby regions of an embedding space (Devlin et al., 2019; Reimers & Gurevych, 2019; Vaswani et al., 2017). Embedding-based topic-modeling systems use this geometry to organize documents before deriving human-readable topic descriptions, which can be useful for short or lexically heterogeneous text (Angelov, 2020; Bianchi et al., 2021).

BERTopic is a modular framework in this family (Grootendorst, 2022). Its default workflow combines contextual embeddings, UMAP dimensionality reduction (McInnes et al., 2018), HDBSCAN clustering (Campello et al., 2013; McInnes et al., 2017), and class-based TF-IDF (c-TF-IDF) topic representations. Researchers can replace these components, but the official implementation and the compatible embedding and clustering ecosystem are maintained primarily in Python.

This dependence on Python introduces an interoperability step for projects whose

data management, statistical analysis, visualization, and reporting are conducted in R (R Core Team, 2026). Packages such as `tm`, `tidytext`, and `quanteda` support R text-analysis workflows (Benoit et al., 2018; Feinerer et al., 2008; Silge & Robinson, 2016); `reticulate` connects these workflows to Python (Ushey et al., 2026). Dedicated interfaces can organize Python selection, argument passing, object conversion, and error diagnosis. For example, `bertopicr` provides helpers for environment setup, training, persistence, inspection, and visualization (Petrič, 2026). Choosing an interface therefore requires attention to its object model, supported backend versions, and evidence for the operations an analysis needs.

We present **BERTopic**, an alternative R interface organized around a retained Python model and familiar S3 methods, including `predict()`, `coef()`, `as.data.frame()`, and `fortify()`. Explicit dense or sparse export and environment diagnostics support subsequent R analyses. A worked example demonstrates this workflow, followed by a controlled comparison of output agreement, runtime, and memory use against equivalent direct Python code. This comparison examines the wrapper’s handling of selected inputs and outputs; it is not a comparison of two topic-modeling algorithms, a feature-completeness study of R packages, or a validation of the substantive topics recovered from the SMS corpus.

2 Computational Background and Scope

2.1 Scope of the R package

The Python library performs document embedding, dimensionality reduction, clustering, vectorization, topic weighting, and optional label refinement. The R package validates inputs, calls Python through `reticulate`, retains the fitted Python object, and converts selected results to R objects (Figure 1). It also provides environment selection, dependency checks, and model persistence. The following pipeline summary locates these interface operations within **BERTopic**; mathematical derivations remain in the cited upstream method papers.

2.2 The six-stage upstream pipeline

The R package delegates computation to the upstream Python library. The default BERTopic workflow can be summarized in six stages:

1. An `embedding_model` converts documents into dense contextual vectors.
2. A `umap_model` or another compatible reducer projects the vectors into a lower-dimensional space.
3. An `hdbscan_model` or another compatible clusterer assigns documents to candidate topics and may identify outliers.
4. A `vectorizer_model` constructs topic-level term counts from the documents assigned to each cluster.
5. A `ctfidf_model` weights terms by their distinctiveness across topic-level document classes.
6. An optional `representation_model` refines, reranks, or relabels the topic representation.

The resulting object combines document assignments, topic-level term representations, fitted component objects, and optional membership-strength information. The outlier topic is conventionally labeled `-1`. BERTopic is not a fully generative word-count model, and its probability output should not be interpreted as an LDA-style posterior topic mixture without additional justification.

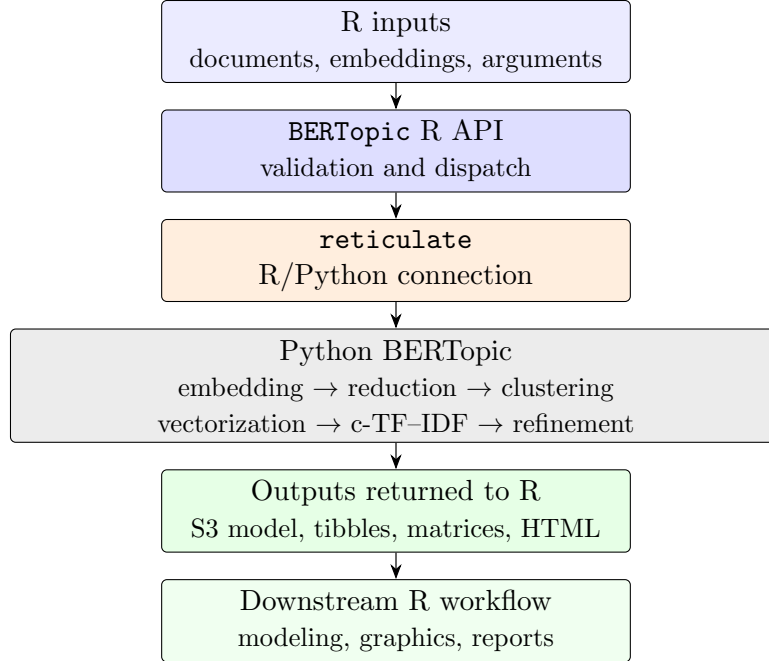


Figure 1: Architecture of the R interface. Core modeling is performed by the official Python library; the R package manages inputs, calls, conversion, and outputs returned to R.

3 Software Design and Implementation

3.1 Relationship to other R interfaces

Both `BERTopic` and `bertopicr` connect R to Python `BERTopic` through `reticulate` and support environment setup, fitting, inspection, persistence, and visualization (Petrič, 2026). The present interface centers these operations on a `bertopicr` S3 object, with explicit conversion to R data structures and environment diagnostics. Table 1 documents this API organization.

The evaluation compares selected operations with direct Python execution. A comparative usability or feature-completeness study would require separate evidence. Backend support, required operations, tests, and preferred object model remain relevant when choosing between R interfaces.

3.2 R wrapper and model object

The main entry point, `bertopic.fit(text, embeddings = NULL, ...)`, calls `BERTopic` from an R session. The function validates that `text` is a character vector, checks that the Python library is available, imports the Python `bertopic` module, and forwards `...` to the Python `BERTopic()` constructor. If the user supplies a numeric R matrix of embeddings, the wrapper converts it to a Python-compatible array and passes it to `fit_transform()`; otherwise, the configured Python embedding model creates the embeddings.

The return value has S3 class `bertopic_r` and contains three main fields: `.py`, the fitted Python `BERTopic` object; `topics`, the document-level topic assignments returned during fitting; and `probs`, the membership-strength matrix when available. Storing `.py` gives users access to the complete fitted Python object. The wrapper converts common results to tibbles, R matrices, sparse matrices, or HTML files. Users can call the Python object directly for methods that do not yet have an R function.

Arguments supplied through `...` are passed to the Python constructor. Users may therefore create Python UMAP, HDBSCAN, vectorizer, c-TF-IDF, or representation objects with `reticulate` and pass them directly to `bertopic.fit()`. This exposes `BERTopic`'s configurable components within the backend versions explicitly supported by the R package. It does not guarantee forward compatibility with changed Python signatures. R also cannot validate every Python argument, so errors in advanced configurations may need to be diagnosed from the Python message and upstream documentation.

The retained Python object has reference semantics, whereas the wrapper's cached R fields are ordinary list elements. Updating topic representations changes the retained Python model in place. Topic reduction also changes assignments and membership strengths; callers must retain the returned wrapper, for example `model <- bertopic_reduce_topics(model, nr_topics = 20L, docs = docs)`, to use the synchronized R fields afterward. This distinction matters for matrix exports, prediction, and persistence.

3.3 Functions and return values

Table 1 summarizes the principal R functions, the Python operation they invoke, and the result returned to R. The functions cover model fitting, inspection, prediction, visualization, modification, and persistence.

Table 1: Mapping Between the R Interface and the Python BERTopic Backend

R function	Python operation	R result or purpose
<code>bertopic_fit()</code>	<code>BERTopic()</code> , <code>fit_transform()</code>	<code>bertopic_r</code> S3 object with retained Python model, assignments, and optional membership strengths
<code>bertopic_topics()</code>	<code>get_topic_info()</code>	Topic metadata as a tibble
<code>bertopic_topic_terms()</code>	<code>get_topic()</code>	Terms and c-TF-IDF weights as a tibble
<code>bertopic_find_topics()</code>	<code>find_topics()</code>	Query-nearest topic identifiers, scores, and labels
<code>bertopic_get_document_info()</code>	<code>get_document_info()</code>	Document-level metadata as a tibble
<code>bertopic_get_representative_docs()</code>	<code>get_representative_docs()</code>	Representative documents as a tibble; <code>top_n</code> limits the R result
<code>bertopic_transform()</code>	<code>transform()</code>	New-document assignments and optional membership strengths
<code>predict()</code>		
<code>bertopic_as_document_topic_matrix()</code>	Stored fitting output	Dense or sparse R matrix for downstream analysis
<code>bertopic_update_topics()</code>	<code>update_topics()</code>	Updated topic representations
<code>bertopic_reduce_topics()</code>	<code>reduce_topics()</code>	Returned wrapper with synchronized assignments and strengths; requires reassignment
<code>bertopic_set_topic_labels()</code>	<code>set_topic_labels()</code>	Custom labels passed with integer topic keys
<code>bertopic_visualize_*()</code>	Python visualization methods	Interactive HTML output
<code>bertopic_save()</code> <code>bertopic_load()</code>	<code>save()</code> , <code>load()</code>	Persistent fitted Python model wrapped for R reuse

S3 methods provide familiar entry points: `print()` and `summary()` summarize the model, `predict()` transforms new documents, `coef()` returns topic terms, and `as.data.frame()` and `fortify()` expose metadata for R workflows. The fitting and extraction comparison in Section 5 and the package checks provide different forms of evidence; the API table alone does not establish equal validation of every helper.

3.4 Dependency management and diagnostics

Because the package calls Python, backend installation is part of normal use. Version 0.1.2 provides Conda and virtualenv helpers and `install_py_deps()` as a dispatcher. Both routes consume the same exact requirements and run the same import validation. The supported tested configuration is Windows, Python 3.10.21, and BERTopic 0.16.0. Full resolved environments are retained with the outputs; other backend versions are outside this support claim.

The installation step is explicit rather than silently triggered by model fitting. Python machine-learning dependencies are large, may download model weights, and may require system-specific resolution. An explicit one-time operation makes this mutation visible. If the backend is missing, the fitting function stops with a diagnostic that names the active Python interpreter and the installation and binding calls required to continue.

Session binding is handled by `use_bertopic()`, `use_bertopic_condaenv()`, and `use_bertopic_virtualenv()`. If `reticulate` has already initialized a different interpreter, these functions stop and instruct the user to restart the R session. This behavior avoids a common failure mode in which package installation succeeds in one environment but the R session imports from another.

Two functions support routine diagnosis. `bertopic_available()` checks importability, while `bertopic_session_info()` reports the interpreter and exact versions for 11 key modules. Full Python distribution versions and R session information are also retained. The documented `bertopic_self_check()` fits, transforms, saves, loads, and compares cached fields, topic metadata, and post-load transformation outputs.

4 Worked Example

4.1 Dataset and example plan

This section demonstrates the main package functions using a 2,247-message subset of the UCI SMS Spam Collection included with the package (Almeida & Hidalgo, 2011). Each row contains a binary label and one English SMS message. The packaged subset contains all 747 spam messages and 1,500 of the 4,827 ham messages in the source collection. **[TODO: Document the rule and random seed, if any, used to select the 1,500 ham messages, and archive a script that reconstructs the packaged data from the UCI source.]** The labels are not used to fit the unsupervised topic model; they are retained to show the data structure and to support optional downstream checks. The direct comparison with Python is reported separately in Section 5.

Table 2: Structure and Descriptive Summary of the Packaged `sms_spam` Data

Variable	R type	Role	Example or summary
<code>label</code>	Character	Reference label; not supplied to BERTopic	1,500 ham (66.8%); 747 spam (33.2%)
<code>text</code>	Character	One document per row; model input	“Nah I don’t think he goes to usf, he lives around here though”
Document length		Characters per message	Median = 84; IQR = 41–147; range = 2–790
Whitespace-delimited length		Descriptive word count	Median = 16; IQR = 8–26; range = 1–162

Note. Length statistics were computed before model fitting. The example text is reproduced from the packaged data.

The example proceeds from environment diagnosis and fitting to inspection, search, prediction, export, visualization, representation updates, and persistence. Its outputs are

archived separately from the frozen-input comparison.

4.2 Environment check and fitting

The following code loads the data, creates the document vector, records the backend configuration, and explicitly fixes UMAP's random state. Setting only the global R or NumPy seed is not sufficient to make every stochastic component deterministic.

```

1  library(BERTopic)
2  library(reticulate)
3
4  stopifnot(bertopic_available())
5  backend <- bertopic_session_info()
6
7  data("sms_spam", package = "BERTopic")
8  docs <- as.character(sms_spam$text)
9
10 set_bertopic_seed(42)
11 sentence_transformers <- reticulate::import("sentence_transformers",
      convert = FALSE)
12 embedding_model <- sentence_transformers$SentenceTransformer(
13   "all-MiniLM-L6-v2", revision = "1110
      a243fdf4706b3f48f1d95db1a4f5529b4d41"
14 )
15 umap <- reticulate::import("umap", convert = FALSE)
16 umap_model <- umap$UMAP(
17   n_neighbors = as.integer(15),
18   n_components = as.integer(5),
19   min_dist = 0,
20   metric = "cosine",
21   random_state = as.integer(42)
22 )
23

```

```

24 model <- bertopic_fit(
25   text = docs,
26   embedding_model = embedding_model,
27   umap_model = umap_model,
28   calculate_probabilities = TRUE
29 )

```

Listing 1: Checking the backend and fitting a BERTopic model from R.

The complete example returned 50 non-outlier topics and 808 outlier messages. It uses the backend’s default vectorizer, whereas the controlled evaluation specifies English stop words. Their term representations and automatic names therefore differ despite matching topic counts. The archived example outputs provide the reference for the code and figure in this section.

4.3 Topic inspection and search

```

1  topic_info <- bertopic_topics(model)
2  head(topic_info[c("Topic", "Count", "Name")], 8)
3
4  valid_topics <- topic_info$Topic[topic_info$Topic != -1]
5  bertopic_topic_terms(model, topic_id = valid_topics[1], top_n = 10)
6  bertopic_get_representative_docs(model, topic_id = valid_topics[1], top_n = 3)
7
8  bertopic_find_topics(
9    model,
10   query_text = "free subscription message",
11   top_n = 5
12 )
13
14 document_info <- bertopic_get_document_info(model, docs)
15 head(document_info, 8)

```

Listing 2: Inspecting and querying the fitted topic solution.

In the retained output, the query `free subscription message` returned topic 7, named `7_comuk.free.unsubscribe.stop`, with a highest similarity score of 0.669. This score measures similarity to a topic representation, not confidence in a behavioral category. Counts, terms, and representative documents provide complementary information for inspecting the match. In the tested backend, `rank` records the returned document order, and `top_n` limits the documents already retained by Python. Researchers should inspect several documents per topic and record any coding or adjudication used to assign substantive labels.

4.4 Prediction and R-native export

```

1 new_docs <- c(
2   "Love you so much, see you tonight.",
3   "Free subscription! Reply STOP to unsubscribe."
4 )
5
6 new_predictions <- predict(model, new_docs, type = "both")
7 new_predictions$topics
8 dim(as.matrix(new_predictions$probs))
9
10 document_topic <- bertopic_as_document_topic_matrix(
11   model,
12   sparse = FALSE,
13   prefix = TRUE
14 )
15 stopifnot(nrow(document_topic) == length(docs))

```

Listing 3: Transforming new documents and exporting training-corpus output.

The two new messages were assigned to topics -1 and 7, respectively. The first result

illustrates that a semantically readable message can still fall outside the fitted density clusters; an outlier assignment is a model result rather than a judgment about the message's meaning. The exported training matrix contained 2,247 rows and 50 non-outlier-topic columns. With `calculate_probabilities=TRUE`, its entries are HDBSCAN-derived membership strengths. Columns carry the actual fitted topic identifiers, whose meanings may change after topic reduction. Dense and sparse exports agreed in the complete example. Using these values as downstream features requires evaluation of their measurement interpretation and uncertainty.

4.5 Visualization, representation updates, and persistence

The wrapper writes Python Plotly figures to HTML and preserves hover information. Its standard HTML export references the Plotly content-delivery network. The retained worked-example outputs therefore also include self-contained HTML, Plotly JSON, and static R figures for offline inspection. Figure 2 illustrates the eight largest non-outlier topics using the full example's fitted assignments and automatic names; it is a static summary rather than a capture of the interactive topic projection.

```

1 bertopic_visualize_topics(
2     model,
3     file = "viz_topics.html"
4 )
5
6 bertopic_visualize_heatmap(
7     model,
8     file = "viz_heatmap.html"
9 )
10
11 # Topic representations can be recomputed after inspection.
12 bertopic_update_topics(model, docs)
13 updated_topic_info <- bertopic_topics(model)
14

```

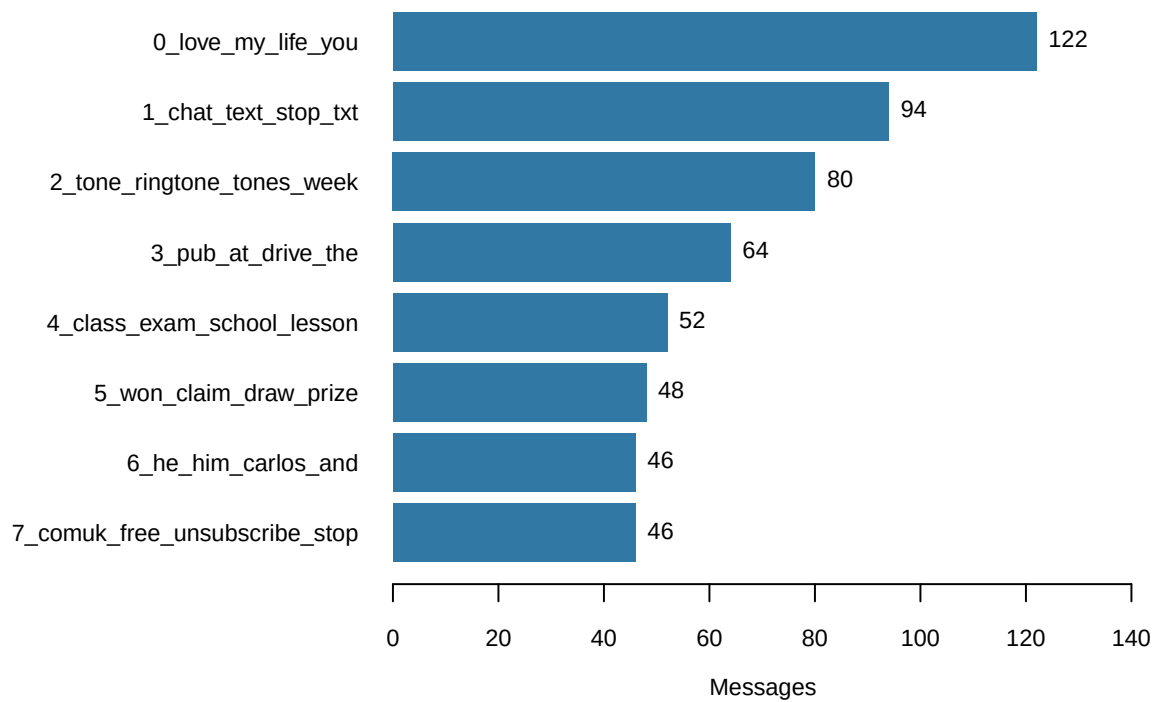


Figure 2: Eight largest non-outlier topics in the complete SMS example, before representation updates. Counts and automatic names come from its archived topic metadata. The controlled benchmark uses a different vectorizer.

```

15 # Save the fitted Python model; load only trusted serialized models.
16 bertopic_save(
17     model,
18     path = "sms_bertopic_model",
19     serialization = "safetensors",
20     overwrite = TRUE
21 )
22 restored_model <- bertopic_load("sms_bertopic_model")
23 restored_topic_info <- bertopic_topics(restored_model)
24 stopifnot(identical(
25     as.data.frame(restored_topic_info)[c("Topic", "Count", "Name")],
26     as.data.frame(updated_topic_info)[c("Topic", "Count", "Name")]
27 ))

```

Listing 4: Creating HTML visualizations and saving the fitted model.

The complete worked example was rerun using the installed 0.1.2 source archive and pinned embedding-model revision. Its console output, numerical tables, HTML, Plotly JSON, and static figures were retained separately from the controlled benchmark. All six restoration checks passed: topic metadata after safetensors loading, and metadata, cached assignments, cached strengths, transformed assignments, and transformed strengths after a full pickle round trip. Lightweight formats omit the fitted reducer and clusterer, so metadata restoration does not establish identical HDBSCAN inference after lightweight loading. Full pickle reuse requires a matching environment and a trusted serialized model.

The SMS data contain no genuine time variable. We therefore do not interpret a topics-over-time analysis for this corpus. The functions `bertopic_topics_over_time()` and `bertopic_visualize_topics_over_time()` are available for datasets with substantively meaningful timestamps.

5 Comparison with the Python Implementation

5.1 Benchmark design

The evaluation asks whether the R fitting and extraction path preserves the outputs of equivalent direct Python code, and quantifies its runtime and memory requirements. Frozen reduced embeddings isolate this path from transformer inference and stochastic dimensionality reduction.

The benchmark used 2,247 SMS messages and installed `BERTopic` 0.1.2, tag v0.1.2, source commit faee106. The exact source archive checksum is retained with the outputs. Embeddings were generated once with all-MiniLM-L6-v2 at immutable revision 1110a243fdf4706b3f48f1d95db1a4f5529b4d41, then reduced from 384 to five dimensions with UMAP and `random_state=42`. The frozen array was stored as float64 in column-major order to match R’s conversion through `reticulate`. Both paths used `BaseDimensionalityReduction()`, `HDBSCAN` with `min_cluster_size=10` and `prediction_data=TRUE`, an English-stop-word `CountVectorizer`, and `calculate_probabilities=TRUE`. Document and array SHA-256 hashes were 2f90a92a...fb036675 and f8169618...92bc3531; full hashes and preparation settings are retained in the manifest.

Five fresh Python processes and five fresh R processes ran on Windows x64 (build 26200), with 8 physical cores, 16 logical processors, and 15.9 GiB of physical memory visible to the operating system. The stack was R 4.4.1, Python 3.10.21, `BERTopic` 0.16.0, `NumPy` 1.26.4, `scikit-learn` 1.4.2, `PyTorch` 2.1.2+cpu, `transformers` 4.47.0, `UMAP` 0.5.5, and `HDBSCAN` 0.8.37. Process order alternated, with R first on odd repetitions. Both paths inherited `PYTHONHASHSEED=42` and identical single-thread settings for OpenMP and common numerical libraries. Components were constructed before timing. Python used `time.perf_counter()`; R used elapsed `proc.time()`. R fit timing includes wrapper validation and conversion. Cold-process time includes startup, imports, input loading, component construction, fitting, and output writing. Windows peak resident memory was

read from `psutil.Process().memory_info().peak_wset`. These are workflow measurements, not a language-neutral microbenchmark.

5.2 Comparison of model outputs

The controlled model produced 50 non-outlier topics and 808 outlier messages (36.0%). Table 3 shows the eight largest topics. Terms are model outputs rather than author-assigned labels and do not validate behavioral constructs.

Table 3: Largest Non-Outlier Topics in the Controlled SMS Model

Topic	Messages	Four highest-weight terms
0	122	love, life, miss, smile
1	94	chat, text, stop, sex
2	80	tone, ringtone, tones, 8007
3	64	pub, drive, car, im
4	52	class, exam, school, teaches
5	48	won, draw, claim, prize
6	46	carlos, hes, says, tyler
7	46	unsubscribe, free, sms, goto

Note. Terms and counts are reproduced from the frozen-input benchmark output. Topic identifiers are arbitrary model labels.

Table 4 reports the paired comparison. All five pairs returned matching 2,247 assignments, 808 outliers, **Topic/Count/Name** metadata in 51 rows (including the outlier topic), and 510 ordered top-ten term keys. Maximum absolute weight and membership-strength differences were $5\text{e-}16$ and $4.86\text{e-}16$, below the 10^{-12} tolerance. Membership-strength matrices had dimensions 2247×50 .

Output agreement supports the tested conversion and extraction path. Independent stochastic fits require a separate stability assessment, described below.

5.3 Runtime and memory use

Table 5 reports medians and interquartile ranges from five fresh processes per interface. Median fitting took 0.630 seconds in Python and 4.810 seconds in R. Paired R-minus-Python

Table 4: Comparison of R and Python Outputs Across Five Paired Runs

Output	Direct Python	R interface	Comparison
Document assignments	2247	2247	Exact in 5/5 pairs
Outlier assignments	808	808	Exact
Topic-information rows	51	51	Exact
Non-outlier topics	50	50	Exact
Ordered top-term keys	510	510	Exact
Maximum c-TF-IDF difference	—	—	5e-16
Membership-strength dimensions	2247×50	2247×50	Exact
Maximum probability difference	—	—	4.86e-16

differences had medians of 4.183 seconds for fitting, 11.400 seconds for cold-process time, and 200.0 MiB for peak memory. Paired-difference medians are summarized separately from medians for each interface.

Table 5: Runtime and Peak-Memory Benchmark

Interface	Fit time, s	Cold-process time, s	Peak RSS, MiB
Direct Python	0.630 [0.630, 0.633]	16.950 [16.870, 16.960]	402.2 [402.1, 402.4]
R interface	4.810 [4.810, 4.830]	28.300 [28.280, 28.410]	602.9 [601.3, 604.9]

Note. Values are medians [interquartile ranges] from five fresh processes. Embedding generation and UMAP reduction were performed once before the timed paired calls. RSS = resident set size.

Direct Python used less time and memory in this configuration. The extra R fit time includes input validation and conversion; cold-process and peak-memory measurements also include the two-runtime workflow. These aggregate measurements cannot attribute the difference to individual operations or predict its size for an end-to-end analysis.

5.4 Reproducibility under stochastic fitting

Matching results in the controlled comparison and reproducibility across independent model fits are different questions. UMAP, approximate neighbor search, transformer execution, and hardware-specific numerical libraries may introduce variation. Topic identifiers can

also change when clusters are reordered, even if their content is similar. For reproducible analyses, users should (a) set R and Python seeds; (b) pass `random.state` explicitly to stochastic components that expose it; (c) record the R and Python package versions and the selected interpreter; (d) save the fitted model; and (e) evaluate topic stability across seeds or reasonable specifications when substantive conclusions depend on the discovered topics. Comparisons across stochastic fits should align topics by their documents or representations rather than assuming that equal numeric identifiers denote equal topics.

5.5 Package checks and archive reconstruction

The exact 0.1.2 source archive passed R `CMD check --no-manual` on the tested Windows configuration with zero errors and zero warnings. The testthat entrypoint executed 159 expectations with no failures, warnings, or skips. The sole package-check NOTE concerned 341 marked UTF-8 strings in the demonstration data. These checks cover the exercised wrapper operations and regression cases; they do not establish support for every Python constructor configuration.

A separate confirmation run cloned the archived Git bundle into a new working directory and installed the same source archive into an isolated R library. It reused the validated Python environment and existing R dependencies, rather than repeating a clean dependency installation. All five paired output comparisons and all six example restoration checks passed; eleven numerical CSV exports from the example matched the formal retained outputs byte for byte. Confirmation timings are archived separately and do not replace the measurements in Table 5. Topic-metadata CSV exports encode list columns as JSON-valued cells; numeric matrix exports retain ordinary CSV columns. An additional checkout with `core.autocrlf=false` verified that the corrected archive preserves the recorded input bytes independently of that Git setting.

6 Availability, Installation, and Operational Guidance

6.1 Software availability

The evaluation used BERTopic 0.1.2, source tag v0.1.2 and commit faee106. Development source and issue tracking are at <https://github.com/Feng-Ji-Lab/BERTopic>; the CRAN project page is <https://CRAN.R-project.org/package=BERTopic>. Reproduction requires the exact retained source archive, rather than an unspecified current repository release. Versioned analysis materials are public under analysis-v0.1.2-r1, including the corrected input archive and reconstruction scripts. A version-specific OSF/Zenodo DOI remains pending. The R interface is MIT-licensed; dependencies and other archived materials retain their respective rights.

6.2 One-time backend setup

All relative paths below refer to a checkout of the package’s analysis materials. For the recorded Python configuration, create a new Windows Conda environment from the archived resolved specification before starting R:

```
1 conda env create --name r-bertopic-paper-012 --file provenance/windows-
   bertopic-0.16.0.yml
```

Listing 5: Creating the recorded Python environment from PowerShell.

Install the retained R source archive and bind the named environment before Python is initialized:

```
1 install.packages("provenance/releases/BERTopic_0.1.2.tar.gz",
2                 repos = NULL, type = "source")
3 library(BERTopic)
4
5 stopifnot(packageVersion("BERTopic") == "0.1.2")
6 use_bertopic_condaenv("r-bertopic-paper-012")
7 stopifnot(bertopic_available())
```

```
bertopic_session_info()
```

Listing 6: Installing the tested R package and binding its Python backend.

If another interpreter has already been initialized by `reticulate`, restart R before binding this environment. For general installation, `install_py_deps_conda()` and the `virtualenv` helper consume the same declared requirements. Those declarations are distinct from the fully resolved Windows environment retained for this analysis; installing them later can select different transitive dependencies.

The benchmark and example session files record R 4.4.1, `reticulate` 1.45.0, and the R dependency versions used. The supplied R dependency installer bootstraps from current CRAN and is not a complete historical R lockfile. An identical R stack therefore requires restoring the recorded versions separately. Backend installation and first-use model downloads can require network access. The pinned embedding-model revision, frozen reduced array, interpreter settings, seeds, full Python distributions, and file hashes are recorded with the outputs.

6.3 Common installation and runtime problems

7 Discussion

The results support the package’s central interface claim: with the recorded frozen inputs and components, the tested R fitting and extraction path preserved selected outputs from direct Python execution. Package regressions and restoration checks provide additional evidence for the exercised workflow operations. The timing and memory differences quantify an integration cost for this short Windows task; they do not measure a complete analysis including transformer inference and UMAP.

The practical benefit is access to the fitted Python model and its common outputs within an R project. Assignments, terms, membership strengths, and visualizations can

Table 6: Common Installation and Runtime Problems

Observed problem	Likely cause	Recommended action
<code>bertopic_available()</code> is false	BERTopic is absent from the active interpreter	Run <code>bertopic_session_info()</code> , verify the Python path, then run <code>install_py_deps()</code>
Requested environment differs from active Python	<code>reticulate</code> initialized too early	Restart R, call <code>use_bertopic()</code> before importing another Python module
Transformer model cannot be downloaded	Network, cache, or model-hub access	Pre-download the model in the bound environment or supply precomputed embeddings
Model fits but results differ across runs	A stochastic component lacks an explicit state	Pass <code>random_state</code> to UMAP or other relevant components and record all versions
Saved model cannot be restored elsewhere	Backend or serialization mismatch	Match versions and test the required post-load operation; lightweight files omit clustering components

feed subsequent R analyses without repeated file exchange. This is useful when preparation, modeling, graphics, and reporting already use R. Python-centered projects may prefer direct access to upstream features and a single runtime. Among R interfaces, users should compare the object model, backend support, documentation, and tests with those of `bertopicr`.

Reproducibility depends on the surrounding software environment. Changed Python signatures, compiled dependencies, model downloads, and hardware-specific libraries can affect execution and results. The present evidence covers BERTopic 0.16.0 on one Windows configuration, one corpus, and five timing replications. The archive confirmation used a new checkout and isolated package installation while reusing existing dependencies; a complete historical R dependency lock remains unavailable. Analyses should record both environments and component settings, retain the source and fitted model, and test the restoration operation they intend to use. Versioned R research packages can help make these dependencies visible (Vuorre & Crump, 2021).

The substantive limits of unsupervised topic modeling are unchanged by the interface. A coherent list of top words does not establish that a topic is a reliable psychological construct. Short documents near cluster boundaries may move between a substantive topic and the outlier class under modest specification changes, and HDBSCAN-derived membership strengths are not automatically calibrated probabilities of theoretically defined categories. Behavioral applications should justify the unit of analysis and embedding model, report preprocessing and component choices, inspect multiple representative documents, assess stability across reasonable specifications, and validate any topic-derived measure against its intended use. Evaluations of behavioral text-mining methods similarly emphasize that convenient model output cannot substitute for task-specific validation against human judgments or other relevant criteria (Macanovic & Przepiorka, 2024).

The interface connects the official Python model to familiar R objects, environment diagnostics, and checked persistence workflows. The controlled comparison supports its tested input–output mappings, while application-specific evidence remains necessary for interpreting

topic-derived behavioral measures.

Declarations

Funding. [TO DO: Insert complete funding statement or “No funding was received for conducting this study.”]

Competing interests. [TO DO: Insert author-approved competing-interests statement.]

Ethics approval. Not applicable. The software demonstration analyzes an existing public dataset distributed with the package.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Availability of data and materials. The exact packaged SMS input, frozen reduced embeddings, preparation and analysis scripts, raw outputs, environment specification, restoration checks, figures, and manuscript materials are retained in the versioned public analysis source. Historical ham-selection reconstruction remains unresolved as noted in the example. [TO DO: Insert the published version-specific OSF/Zenodo archive DOI.]

Code availability. Development source is at <https://github.com/Feng-Ji-Lab/BERTopic>. The analyses used version 0.1.2, source tag v0.1.2, commit faee106, and the retained source archive identified by SHA-256 in the release record. Analysis scripts and provenance are versioned separately under analysis-v0.1.2-r1. [TO DO: Insert the published version-specific archive DOI for the exact source and analysis scripts.]

Author contributions. [TO DO: Complete an author-approved CRediT contribution statement. Ensure that manuscript authorship and package-development contributions are documented consistently.]

Open Practices Statement

The package source, inputs used in the reported analyses, scripts, raw benchmark and example outputs, tables and figures, environment records, and reconstruction documentation are available as versioned GitHub analysis materials. The confirmation run reused an existing validated dependency environment, and the historical SMS selection mechanism is not yet reconstructable from the original source. A durable OSF/Zenodo DOI remains to be supplied. The software evaluation was not preregistered.

References

- Almeida, T., & Hidalgo, J. (2011). SMS spam collection. <https://doi.org/10.24432/C5CC84>
- Angelov, D. (2020). Top2Vec: Distributed representations of topics. <https://doi.org/10.48550/arXiv.2008.09470>
- Benoit, K., Watanabe, K., Wang, H., Nulty, P., Obeng, A., Müller, S., & Matsuo, A. (2018). quanteda: An R package for the quantitative analysis of textual data. *Journal of Open Source Software*, 3(30), 774. <https://doi.org/10.21105/joss.00774>
- Bianchi, F., Terragni, S., & Hovy, D. (2021). Pre-training is a hot topic: Contextualized document embeddings improve topic coherence. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, 759–766. <https://doi.org/10.18653/v1/2021.acl-short.96>
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent dirichlet allocation. *Journal of Machine Learning Research*, 3(Jan), 993–1022. <https://www.jmlr.org/papers/v3/blei03a.html>
- Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in knowledge discovery and data mining* (pp. 160–172). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-37456-2_14

- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391–407. [https://doi.org/10.1002/\(SICI\)1097-4571\(199009\)41:6<391::AID-ASI1>3.0.CO;2-9](https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9)
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Feinerer, I., Hornik, K., & Meyer, D. (2008). Text mining infrastructure in R. *Journal of Statistical Software*, 25(5), 1–54. <https://doi.org/10.18637/jss.v025.i05>
- Grimmer, J., & Stewart, B. M. (2013). Text as data: The promise and pitfalls of automatic content analysis methods for political texts. *Political Analysis*, 21(3), 267–297. <https://doi.org/10.1093/pan/mps028>
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. <https://doi.org/10.48550/arXiv.2203.05794>
- Hofmann, T. (1999). Probabilistic latent semantic indexing. *Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 50–57. <https://doi.org/10.1145/312624.312649>
- Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755), 788–791. <https://doi.org/10.1038/44565>
- Macanovic, A., & Przepiorka, W. (2024). A systematic evaluation of text mining methods for short texts: Mapping individuals’ internal states from online posts. *Behavior Research Methods*, 56, 2782–2803. <https://doi.org/10.3758/s13428-024-02381-9>
- McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11), 205. <https://doi.org/10.21105/joss.00205>

- McInnes, L., Healy, J., Saul, N., & Großberger, L. (2018). UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29), 861. <https://doi.org/10.21105/joss.00861>
- Petrič, T. (2026). *bertopicr: Topic modeling with BERTopic* [R package version 0.3.6]. <https://doi.org/10.32614/CRAN.package.bertopicr>
- R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://doi.org/10.32614/R.manuals>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Roberts, M. E., Stewart, B. M., Tingley, D., Lucas, C., Leder-Luis, J., Gadarian, S. K., Albertson, B., & Rand, D. G. (2014). Structural topic models for open-ended survey responses. *American Journal of Political Science*, 58(4), 1064–1082. <https://doi.org/10.1111/ajps.12103>
- Silge, J., & Robinson, D. (2016). tidytext: Text mining and analysis using tidy data principles in R. *Journal of Open Source Software*, 1(3), 37. <https://doi.org/10.21105/joss.00037>
- Ushey, K., Allaire, J. J., & Tang, Y. (2026). *reticulate: Interface to Python* [R package version 1.46.0]. <https://doi.org/10.32614/CRAN.package.reticulate>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Vuorre, M., & Crump, M. J. C. (2021). Sharing and organizing research products as R packages. *Behavior Research Methods*, 53, 792–802. <https://doi.org/10.3758/s13428-020-01436-x>